circl.

Hyper-Local Resource Sharing & Microservices Ecosystem

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title:	CIRCL Mobile Platform
Document Version:	1.0.0 (Final Baseline)
Date:	July 15, 2026
Author / Organization:	Software Engineering Team
Target Platform:	iOS, Android & Web Admin

Table of Contents

1. Introduction
    1.1 Purpose
    1.2 Scope
    1.3 Definitions, Acronyms, and Abbreviations
2. Overall Description
    2.1 Product Perspective
    2.2 Product Functions
    2.3 User Characteristics
    2.4 Assumptions and Dependencies
3. External Interface Requirements
    3.1 User Interfaces
    3.2 Hardware Interfaces
    3.3 Software Interfaces
4. Functional Requirements
    FR1: Authentication, Security & Identity Verification
    FR2: Listing, Request & Offer Management
    FR3: Geospatial Search & Interactive Mapping
    FR4: Real-Time Messaging & Chat System
    FR5: Escrow Payment & Financial Transactions
    FR6: Aura Reputation & Trust Economy
    FR7: Dispute Resolution & Escalation Center
    FR8: Platform Moderation & Admin Analytics
5. Non-Functional Requirements
    5.1 Performance Requirements
    5.2 Security & Privacy Requirements
    5.3 Usability & Design Requirements
    5.4 Reliability, Availability & Maintainability
6. System Requirements
    6.1 Hardware Requirements
    6.2 Software Requirements

6.3 Tools & Technologies Stack

7. Use Case Analysis & UML Diagrams
   7.1 System Use Case Diagram (UML)
   7.2 Detailed Use Case Specifications (UC1 - UC4)

8. Constraints & Future Enhancements
   8.1 System Constraints
   8.2 Future Enhancements Roadmap

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a complete, definitive, and detailed description of the operational requirements, behavioral logic, hardware/software interfaces, and systemic performance parameters of **CIRCL** (A Smart Community Platform for Resource Sharing and Microservices). This specification serves as the formal baseline for developers, system architects, test engineers, project managers, and quality assurance auditors.

1.2 Scope

CIRCL is a cross-platform mobile application ecosystem designed to solve community hyper-isolation and material underutilization in suburban and urban settings. It creates a hyper-local dual-pillared marketplace that enables neighbors to share physical resources (goodwill borrowing and monetized tool/equipment rentals) and collaborative micro-services (volunteer favours and paid gig assignments).

> **Key Capabilities Covered by Scope:** Automated National Identity Card (NIC) verification via Google ML Kit OCR and SHA-512 duplicate hashing; location-based radius discovery using Google Maps; offer & counter-proposal bargaining; PayHere mobile payment gateway with safety deposit holds; real-time WebSocket messaging; Aura Points gamification; and a web-enabled administrative dispute arbitration dashboard.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition & Context
SRS	Software Requirements Specification document.
CIRCL	Smart Community Platform for Resource Sharing and Microservices.
BaaS	Backend-as-a-Service (Supabase platform supplying DB, Auth, Realtime, Storage).
NIC	National Identity Card (Government-issued photographic identity document).
OCR	Optical Character Recognition (Google ML Kit on-device string extraction engine).
Aura Points	Platform trust score awarded for verified transactions, positive ratings, and community identity verification.
Escrow	Financial structure temporarily holding security deposits during tool hire to guarantee return.
RLS	Row-Level Security (PostgreSQL data access control policies evaluated per user session).
RPC	Remote Procedure Call (Stored procedure functions in PostgreSQL database tier).
PayHere	Central Bank approved mobile payment gateway integrated for credit card processing in LKR.

2. Overall Description

2.1 Product Perspective

CIRCL is a **new, standalone mobile and cloud software ecosystem**. The client layer is built using Flutter for cross-platform iOS and Android execution. The server tier relies on Supabase BaaS providing PostgreSQL database management, realtime WebSocket notifications, file storage buckets, and identity authentication.



2.2 Product Functions

The primary functionality of the system is partitioned into eight main modules:

- **User Auth & Security Module:** Email/Password login, Google SSO, App Lock PIN/Biometrics, and ML Kit NIC document parsing.
- **Listing & Marketplace Module:** Creation and filtering of goodwill borrowing, monetized rentals, favours, and micro-gigs.

- **Map & Geolocation Module:** Pin location rendering, geodesic distance calculations, and distance radius filtering.
- **Negotiation Engine:** Offer submissions, counter-proposals, deposit agreements, and request locking.
- **Real-Time Messaging System:** Instant chat coordination, embedded negotiation cards, image sharing, and handover confirmations.
- **Payment & Escrow Safety:** PayHere card processing, pre-authorized safety deposit holds, 4% platform fee calculation, and deposit releases.
- **Aura Trust Economy:** Points calculation, levelling up (Bronze to Platinum), rating feedback, and deposit tier discounts.
- **Admin Governance Dashboard:** Financial transaction charts, dispute arbitration, OCR manual review queue, and moderation controls.

2.3 User Characteristics

- **Community Borrower / Requester:** Smartphone user seeking localized tools or favours. Requires simple UI flows and rapid local search.
- **Community Lender / Provider:** Resident seeking to monetize idle appliances or offer service skills. Requires transparent escrow protections.
- **Platform Administrator:** Staff handling operations, inspecting disputed cases, executing deposit refunds/forfeitures, and enforcing community safety rules.

2.4 Assumptions and Dependencies

- Active cellular data or Wi-Fi network availability on mobile client hardware.
- Integrated operational GPS receiver and camera hardware on client mobile devices.
- Continuous operational availability of Supabase Cloud services, Google Maps APIs, and PayHere gateway servers.

3. External Interface Requirements

3.1 User Interfaces

The CIRCL user interface is designed using Flutter with modern dark-mode glassmorphism design tokens (#26A69A Teal, #1E293B Card Slate, #070913 Background). Key interfaces include:

- **Splash & Onboarding Screen:** Interactive app overview carousel with page indicators.
- **Auth & Lock Screens:** Tabs for login/signup, OAuth buttons, PIN entry keypads, and native biometric prompts.
- **Home Feed & Map View:** Responsive listing grids with category pills and an embedded Google Maps view displaying custom pin markers.
- **Post Creation Form:** Step-by-step wizard for listing details, goodwill vs. monetized rate selector, and photo attachments.
- **Real-time Chat Screen:** Timeline interface showing message bubbles, embedded interactive offer cards, photo sharing buttons, and transaction status banners.
- **Payment Checkout UI:** Wallet summary, itemized fee breakdown, and PayHere native webview checkout bridge.
- **Admin Dispute Dashboard:** Data analytics summary, tabbed dispute resolution table, manual NIC review list, and moderation buttons.

3.2 Hardware Interfaces

- **Camera Sensor:** Used by camera plugins to capture photographic images of identity cards and listing items.
- **Biometric Scanner:** TouchID / FaceID hardware sensors bound via native local authentication libraries.
- **GPS Receiver:** Hardware location provider yielding precise latitude/longitude positioning data.

3.3 Software Interfaces

- **Supabase API:** HTTPS REST & WSS protocol endpoints for database transactions, storage buckets, and realtime subscriptions.
- **Google Maps API:** Native Android/iOS mapping render engine for visual geographic discovery.
- **PayHere Mobile SDK:** Encrypted payment gateway integration for processing credit/debit card authorizations.
- **Google ML Kit Text Recognition:** On-device native SDK parsing text strings from image buffers.

4. Functional Requirements

FR1: Authentication, Registration & Access Control

- **FR1.1: Email Registration & OTP Verification:** The system shall allow users to register with an email and password, requiring account activation via a 4-digit OTP.
- **FR1.2: Google OAuth Authentication:** The system shall allow users to sign up and log in via Google OAuth, automatically creating a profile if one does not exist.
- **FR1.3: Credential Complexity & Uniqueness:** The system shall enforce password complexity rules (8–15 characters, mixed case, numbers, symbols) and validate username uniqueness in real time.
- **FR1.4: Mandatory Terms & Safety Acknowledgement:** The system shall require users to accept the Terms of Conditions and Safety Guidelines before completing registration.
- **FR1.5: Login & Session Persistence:** The system shall authenticate users via email/password and provide a "Remember Me" toggle for session persistence.
- **FR1.6: Biometric Authentication Setup:** The system shall allow users to enable Face ID or device biometrics to authorize secure actions like payments.
- **FR1.7: Password Recovery:** The system shall allow users to reset forgotten passwords via a secure email recovery link.
- **FR1.8: Role-Based Routing:** The system shall check user roles upon login, routing administrative users to the Admin Dashboard and regular users to the Feed Screen.
- **FR1.9: Secure Logout:** The system shall allow users to securely log out and clear locally stored authentication tokens.

FR2: User Profiles, Gamification & Verification

- **FR2.1: Automatic Profile Provisioning:** The system shall create a default profile upon registration, initializing username, location, avatar, Circl Points (0), and Aura Level (Bronze).
- **FR2.2: Profile Management:** The system shall allow users to update display names, locations, contact info, and inventory lists.
- **FR2.3: Onboarding & Profile NIC Verification:** The system shall prompt first-time users with a National Identity Card (NIC) verification step via secure hashing, allowing them to skip it during onboarding and complete it later in their profile settings.
- **FR2.4: Verification Badge Display:** The system shall display a verification badge on profiles and request cards for NIC-verified users.
- **FR2.5: Circl Points Award System:** The system shall automatically award Circl Points to users upon completing transactions.

- **FR2.6: Aura Level Progression & Display:** The system shall automatically upgrade a user's Aura Level (Bronze, Silver, Gold, Diamond) as their Circl Points reach defined mathematical thresholds, and visually display this tier on their avatar.

FR3: Resource Request Creation & Discovery

- **FR3.1: Multi-Type Request Creation:** The system shall allow users to create four request types: Free Borrow, Paid Rent, Free Favour, and Paid Task.
- **FR3.2: Guided Creation Workflow:** The system shall guide users through a multi-step request creation process covering task details, location preferences, and a final review.
- **FR3.3: Dynamic Financial & Duration Inputs:** The system shall dynamically display input fields for expected duration, budget limits, and security deposits based on the selected request type.
- **FR3.4: Location Preferences & GPS Detection:** The system shall allow users to specify execution preferences (In-Person, Flexible, or Remote) and auto-detect coordinates using device GPS.
- **FR3.5: Address Autocomplete & Saved Locations:** The system shall provide real-time address autocomplete suggestions and allow users to save frequently used locations.
- **FR3.6: Request Lifecycle & Expiration:** The system shall allow users to edit or cancel open requests, automatically expiring unaccepted requests after 7 days.
- **FR3.7: Feed & Dynamic Filtering:** The system shall display open requests in a scrollable feed with real-time filters for type, distance, and price.
- **FR3.8: Keyword Search Engine:** The system shall provide a search bar to query open requests by matching keywords in titles and descriptions.
- **FR3.9: Interactive Map Display:** The system shall display an interactive map with colour-coded markers representing different request types.
- **FR3.10: Intelligent Matching Algorithm:** The system shall rank requests using a compatibility score based on spatial proximity, user ratings, and Circl Points, highlighting the best match.

FR4: Transactions, Negotiation & Escrow

- **FR4.1: Request Acceptance & Transaction Setup:** The system shall allow users to accept open requests, changing status to "In Progress" and creating a transaction record.
- **FR4.2: Payment Gateway Integration:** The system shall integrate a payment gateway to facilitate secure payments, including simulated escrow holds for Rent and Paid Task requests.
- **FR4.3: Mutual Transaction Completion:** The system shall require both transaction parties to independently confirm completion before finalizing the exchange and releasing escrowed funds.

- **FR4.4: Dispute Management:** The system shall allow users to flag transactions for dispute, freezing simulated deposits, ratings, and points until an administrator resolves it via the Admin Dashboard.

FR5: Real-Time Communication

- **FR5.1: Chat Hub Management:** The system shall provide a centralized chat inbox allowing users to search, filter (Unread, Active, Completed), and sort conversations.
- **FR5.2: Real-Time Peer-to-Peer Chat:** The system shall open a private, real-time messaging channel between users during negotiation and active transaction phases.
- **FR5.3: Rich Media Attachments:** The system shall allow users to securely transmit images and documents (PDF/Word) within the chat interface.
- **FR5.4: Automated System Messaging:** The system shall automatically inject contextual system events into the chat feed (e.g., "Payment completed", "Terms accepted", "Cancellation requested").
- **FR5.5: In-App Notification Engine:** The system shall send real-time in-app notifications for incoming messages, request acceptances, and transaction status updates.

FR6: Ratings, Community & Moderation

- **FR6.1: Post-Transaction Ratings & Reviews:** The system shall prompt both parties to submit a 1-to-5 star rating and optional text review upon transaction completion.
- **FR6.2: Automated Rating Aggregation:** The system shall automatically recalculate and update user average ratings and review counts upon new submissions.
- **FR6.3: User Moderation & Reporting:** The system shall allow users to report other profiles for spam, harassment, or suspicious activity.

FR7: System Preferences & Ecosystem

- **FR7.1: Multi-Language Support:** The system shall support real-time dynamic language switching between Tamil, Sinhala, and English.
- **FR7.2: Theme Adaptability:** The system shall support Light and Dark themes, adapting dynamically to device settings.
- **FR7.3: Companion Web Portal Integration:** The system shall provide a companion website displaying app features, platform info, and Android APK downloads.

5. Non-Functional Requirements

5.1 Performance Requirements

- **Response Time:** Database query response times and REST requests shall execute within 2.0 seconds over standard 4G connections.
- **Chat Latency:** Messages delivered over Supabase Realtime WebSockets shall display on recipient screens within 500 ms.
- **Frame Rate:** Map interface rendering shall maintain 60 FPS performance when displaying up to 100 pin markers simultaneously.

5.2 Security & Privacy Requirements

- **Encryption in Transit:** All web traffic shall be encrypted using TLS 1.3 protocol standards over HTTPS and WSS connections.
- **Data Access Rules:** PostgreSQL table access shall be guarded by Row-Level Security (RLS) policies based on authenticated user IDs.
- **Identity Protection:** NIC numbers shall be stored strictly as salted SHA-512 hashes; plain text identity numbers shall never be written to permanent storage.
- **Keystore Security:** Auth tokens and app lock credentials shall be stored in native device hardware keychains via `flutter_secure_storage`.

5.3 Usability Requirements

- **Design System:** UI components shall adhere to dark theme glassmorphic styling utilizing clean typography and clear contrast ratios.
- **Interaction Efficiency:** Core tasks (creating a request, accepting an offer, entering chat) shall require no more than 3 user taps from the primary feed screen.
- **Localization Readiness:** App text strings shall be organized inside localizable translation bundles (`l10n.yaml`).

5.4 Reliability & Availability

- **System Availability:** The backend server database environment shall maintain 99.9% uptime.
- **Fault Recovery:** Mobile clients shall gracefully handle internet drops by queueing unsent messages locally until connection restoration.

6. System Requirements

6.1 Hardware Requirements

Component	Minimum Specifications	Recommended Specifications
Mobile Processor	Quad-Core 1.8 GHz ARM64	Octa-Core 2.4 GHz ARM64
Mobile RAM	3 GB RAM	4 GB RAM or higher
Storage	150 MB free space	500 MB free space
Camera & Location	5 MP Rear Camera, GPS Receiver	12 MP Camera, High-Precision GNSS
Cloud Server Instance	Multi-core virtualized DB instance	Managed Supabase Pro Instance (4GB+ RAM)

6.2 Software Requirements

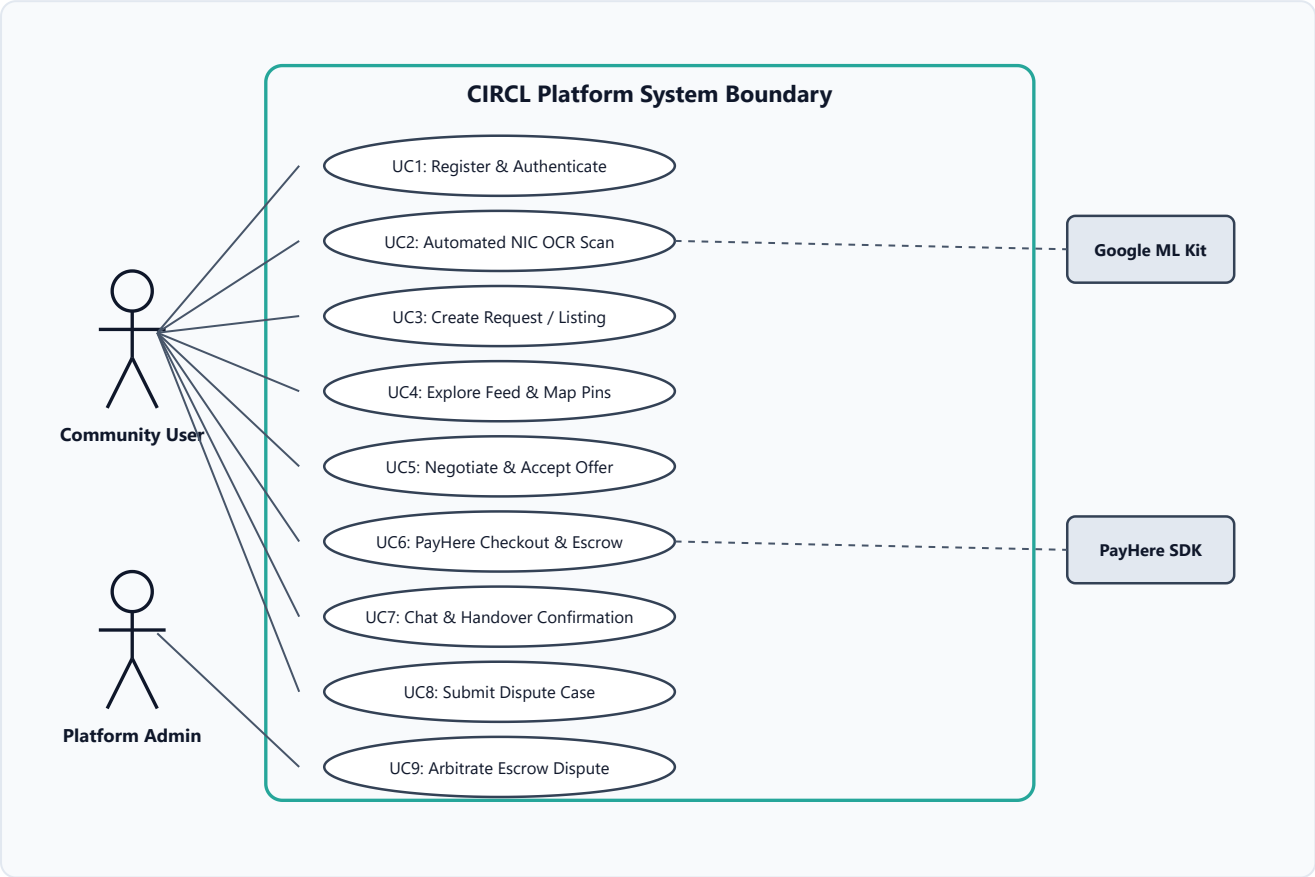
- **Mobile OS:** Android 8.0 (API Level 26) or higher / iOS 13.0 or higher.
- **Admin Platform:** Windows 10/11, macOS, or Linux running modern web browsers (Chrome, Edge, Firefox).
- **Database Engine:** Managed PostgreSQL 15 with PostGIS extensions.

6.3 Tools & Technologies Stack

Layer	Technology / Package	Version & Role
Frontend Framework	Flutter / Dart SDK	^3.11.0 (Cross-platform client framework)
Backend Platform	Supabase Cloud	PostgreSQL 15, Auth, Storage, WebSockets
Payment Gateway	PayHere Mobile SDK	^3.2.3 (LKR credit card checkout & escrow)
Map Services	Google Maps Flutter & Geolocator	`google_maps_flutter` ^2.11.0, `geolocator` ^14.0.3
OCR Scanner Engine	Google ML Kit Text Recognition	`google_mlkit_text_recognition` ^0.15.1
Local Auth Security	Flutter Secure Storage & Local Auth	`flutter_secure_storage` ^10.3.1, `local_auth` ^3.0.1
Cryptography	Crypto Package	^3.0.3 (SHA-512 identity hashing)

7. Use Case Analysis & UML Diagrams

7.1 System Use Case Diagram (UML)



7.2 Detailed Use Case Specifications

UC1: Create Resource Request or Listing

Use Case ID & Name	**UC1:** Create Resource Request or Listing
Primary Actor	Community User (Requester / Lender)
Preconditions	User must be registered, authenticated, and logged into an active account session.
Trigger	User taps the floating action button (+) on the main home screen.
Main Flow	1. User selects classification: Physical Resource (Borrow/Rent) or Service (Favour/Gig). 2. User fills in title, detailed description, category, and target location. 3. User specifies pricing mode (Free Goodwill vs. Rental Rate & Escrow Deposit). 4. User uploads item photographs via camera or gallery picker. 5. User taps "Publish Listing". 6. System validates inputs, uploads photos to Supabase Storage, and inserts a row into `requests` table with `status = 'open'`. 7. System renders success confirmation and renders post on feed and map.
Alternative Flows	*4a. Missing Fields:* System displays inline red validation text highlighting missing inputs. *6a. Storage Failure:* System alerts user of network failure and allows immediate retry.
Postconditions	Listing becomes searchable and visible on nearby user feeds and map pins.

UC2: Accept Request & Complete Escrow Transaction

Use Case ID & Name	**UC2:** Accept Request and Escrow Payment Settlement
Primary Actor	Community Provider / Borrower
Secondary Actor	PayHere Payment Gateway SDK
Preconditions	User is logged in, NIC verified, and viewing an open listing posted by another resident.
Trigger	User taps "Accept / Propose Offer" button on listing view.
Main Flow	1. User inputs proposed rate and deposit, sending an offer. 2. Requester reviews offer in real-time chat and taps "Accept Offer". 3. System inserts transaction row and locks request status to `negotiating`. 4. Borrower proceeds to wallet checkout screen. 5. System calculates total payable (Rate + Safety Deposit Hold + 4% Platform Fee). 6. System invokes PayHere SDK checkout dialog for card details. 7. PayHere confirms card authorization. 8. System logs payment event, holds deposit in escrow table, and updates transaction to `in_progress`.
Alternative Flows	*7a. Card Declined:* PayHere returns authorization error. System retains `negotiating` lock state and prompts user to re-attempt with another payment card.
Postconditions	Deposit is safely held in escrow, transaction transitions to active execution, and handover timeline renders in chat.

UC3: Verify Identity via Automated NIC OCR

Use Case ID & Name	**UC3:** Automated NIC OCR Document Verification
Primary Actor	Community User
Secondary Actor	Google ML Kit Text Recognition Engine
Preconditions	User navigates to Identity Verification screen.
Trigger	User selects "Scan National Identity Card".
Main Flow	1. User positions document within camera scanning box. 2. Scanner captures high-resolution photo buffer. 3. App passes buffer to Google ML Kit OCR engine. 4. System parses document text to extract identity card string. 5. System calculates salted SHA-512 digest hash of the NIC number. 6. System calls `check_duplicate_nic` database function. 7. System confirms unique identity, logs verification record, and marks profile as `verified`.
Alternative Flows	*6a. Duplicate Hash:* System halts verification and alerts user that NIC is already registered to another user. *4a. Blurred Image:* System prompts user to retake scan or queue for manual admin review.
Postconditions	User receives verified profile badge, enabling high-deposit monetized transactions.

UC4: Arbitrate Transaction Dispute (Admin)

Use Case ID & Name	**UC4:** Arbitrate Transaction Dispute & Escrow
Primary Actor	Platform Admin / Moderator
Preconditions	Disputed transaction case logged with status `disputed` by a transacting party.
Trigger	Admin selects dispute record on Admin Dashboard panel.
Main Flow	1. Admin reviews dispute details, uploaded evidence photos, and chat conversation log. 2. Admin selects verdict: Refund Deposit to Borrower OR Forfeit Deposit to Lender. 3. Admin clicks "Execute Dispute Verdict". 4. System invokes ledger RPC stored procedures to settle funds. 5. System dispatches payout to designated party minus platform commission. 6. System updates transaction status to `resolved` and notifies both users.
Postconditions	Escrow ledger is balanced and closed, dispute is marked resolved, and funds are disbursed.

8. Constraints & Future Enhancements

8.1 System Constraints

- **External API Services:** Operational system continuity depends on third-party uptime (Google Maps, Supabase, and PayHere APIs).
- **Network Dependency:** Application logic requires continuous cellular data or Wi-Fi connectivity for financial escrow and chat syncing.
- **Regulatory Compliance:** Online payment settlement logic strictly adheres to Central Bank monetary rules executed via PayHere gateway protocols.

8.2 Future Enhancements Roadmap

- **Smart Contract Proof of Return:** Integration of immutable ledger checkpoints for recording item check-in and check-out physical conditions.
- **AI Fair Price Recommender:** Machine learning pricing algorithm suggesting fair daily rental rates based on neighborhood market data.
- **Private Community Circles:** Hyper-local gated sub-communities for universities, housing complexes, and corporate campuses.
- **Integrated Courier Dispatch:** On-demand local delivery API integration to transport heavy equipment directly between users.